

CHAPTER 1

INTRODUCTION

Welcome to the Library Management System:

Managing a library efficiently in today's fast-paced environment requires more than manual record-keeping. The Library Management System is an advanced software solution designed to simplify and automate the daily operations of libraries in educational institutions and public settings. By replacing traditional methods with a computerized approach, the system ensures efficiency, accuracy, and reliability across all library processes.

Purpose and Functionality: Organized and Secure Management

The primary objective of this system is to provide a well-structured and secure platform for managing books, members, and transactions. Every core function, from adding new books to issuing and returning borrowed items, is handled digitally. This minimizes paperwork, reduces human error, and ensures that records remain consistent, accurate, and readily accessible at all times.

Access Levels: Administrator and Member Roles

The Library Management System operates with two distinct access levels:

Administrator Access: Administrators are responsible for maintaining the book database, updating records, managing stock levels, and supervising all transactions. The system allows administrators to track book availability and monitor member activity efficiently.

Member Access: Members can register, log in, search for books, borrow available titles, and return them within the specified due period. All member actions are automatically recorded, ensuring transparency and real-time updates.

Implementation Technologies:

This project is developed using Python as the core programming language and MySQL as the backend database. Python offers flexibility and modularity for handling multiple operations, while MySQL ensures secure, structured, and reliable data storage with strong integrity controls.

Real-World Impact: Efficiency Through Automation

The Library Management System demonstrates strong real-world applicability for libraries and educational institutions. By automating routine tasks and eliminating manual record-keeping, the system significantly improves operational efficiency, data accuracy, and overall user satisfaction. It showcases how technology and automation can transform traditional library environments into modern, streamlined systems.

CHAPTER 2

OBJECTIVES OF THE PROJECT

The main goal of this project is to make library management easier, faster, and error-free using a computer program. Below are the detailed objectives of my Library Management System project:

1. To Digitize Library Records: Replace the old manual system with a digital system where all book and student details are stored on the computer.
2. To Save Time: Reduce the time spent on tasks like searching for a book, issuing it, or checking return dates.
3. To Avoid Human Errors: Eliminate mistakes that can happen while writing records by hand, like issuing the same book to two students or forgetting due dates.
4. To Make Searching Easy: Allow librarians and students to quickly search for books by name, author, or ID using a simple interface.
5. To Track Book Availability: Keep an updated list of available and issued books so that students know which books are free to borrow.
6. To Manage Issuing and Returning: Help the librarian easily issue and return books and keep track of who borrowed what and when.
7. To Use Less Paper: Promote eco-friendly digital record-keeping by reducing the use of registers and paper files.
8. To Provide a Simple User Interface: Design the system so that even those with basic computer knowledge can use it easily without confusion.

CHAPTER 3

PROPOSED SYSTEM

The proposed Library Management System is a fully computerized solution designed to overcome the limitations of the existing manual library system. It aims to provide a fast, reliable, and user-friendly platform for managing library operations efficiently. By automating routine tasks, the system minimizes human intervention, reduces errors, and ensures accurate maintenance of records.

In the proposed system, all library data such as book details, member information, issue records, and return history are stored in a centralized database. This enables quick retrieval of information and ensures data consistency and security. Librarians can easily add, update, or remove book records, monitor book availability, and manage member activities through a secure administrative interface.

The system provides separate access levels for administrators and members. Administrators are responsible for maintaining the database, supervising transactions, and generating reports. Members can register, log in, search for books, borrow and return books, and view their borrowing history. All transactions are recorded automatically, providing real-time updates and transparency.

The proposed system significantly reduces paperwork and manual effort while improving operational efficiency. It ensures better resource management, faster processing of transactions, and improved user satisfaction. Overall, the proposed Library Management System offers a scalable and efficient solution suitable for modern libraries and educational institutions.

CHAPTER 4

SYSTEM DEVELOPMENT LIFE CYCLE (SDLC)

The System Development Life Cycle (SDLC) is a systematic and well-defined framework used for the development of high-quality software systems. It provides a step-by-step approach to designing, developing, testing, and deploying software in an organized manner. By following SDLC, software projects can be completed efficiently while maintaining accuracy, consistency, and reliability throughout the development process.

For the Library Management System, SDLC plays a crucial role in ensuring that the project is developed according to user requirements and real-world library needs. It helps in clearly defining system objectives, identifying functional and non-functional requirements, and designing a structured solution. This approach minimizes errors, improves system performance, and results in a reliable and maintainable application.

The SDLC followed for this project consists of six major phases: Planning, Requirements Analysis, System Design, Implementation, Testing, Deployment, and Maintenance. Each phase contributes to the successful development of the system by addressing specific tasks and goals. Together, these phases ensure that the Library Management System is robust, user-friendly, and capable of supporting efficient library operations.

PHASES OF SYSTEM DEVELOPMENT LIFE CYCLE



INITIATION PHASE

The Initiation Phase for the Library Management System begins when the need to improve or replace the existing manual library processes is identified. Traditional methods of maintaining book records, member details, and transactions are inefficient and prone to errors. This phase focuses on recognizing the opportunity to introduce an automated system that enhances accuracy, efficiency, and overall library operations.

The purpose of the Initiation Phase is to:

- ✓ Identify and validate the need to improve library operations by automating book management, member records, and issue–return processes.
- ✓ Identify key assumptions and constraints, such as budget limitations, availability of technical resources, staff readiness, and infrastructure requirements.
- ✓ Explore alternative solutions to the identified problem, including improving existing manual procedures or implementing a fully computerized Library Management System.
- ✓ Ensure executive and technical sponsorship for the project. A project sponsor appoints a Project Manager, and the business need is formally documented in a Concept Proposal.
- ✓ Align the proposed system with organizational goals, infrastructure, and strategic plans to ensure long-term sustainability and effectiveness.

The Concept Proposal outlines the current library processes, highlights existing issues, and explains how the proposed system supports institutional objectives. Upon approval of the proposal, a Project Management Charter is created, granting authority to initiate the project. Careful oversight during this phase ensures that the Library Management System aligns with strategic objectives and that organizational resources are effectively utilized.

SYSTEM CONCEPT DEVELOPMENT PHASE

The System Concept Development Phase for the Library Management System begins after the business need for automating library operations is formally validated by

organizational leadership and technical authorities. This phase focuses on analyzing possible system solutions and determining the most feasible and effective approach to meet the identified library requirements.

The purpose of the System Concept Development Phase is to:

- ✓ Determine the feasibility and suitability of alternatives such as continuing manual processes, partial automation, or implementing a complete Library Management System.
- ✓ Identify system interfaces including user interfaces for administrators and members, as well as database and application interfaces.
- ✓ Identify basic functional and data requirements required to manage books, members, issue–return transactions, and record maintenance.
- ✓ Establish system boundaries by defining system goals, objectives, critical success factors, and performance measures relevant to library operations.
- ✓ Evaluate costs and benefits of alternative approaches to ensure the selected solution is cost-effective and efficient.
- ✓ Assess project risks such as data security issues, technical limitations, and user adaptability.
- ✓ Identify and initiate risk mitigation actions, and develop a high-level technical architecture, process models, data models, and a concept of operations.

This phase explores potential technical solutions within the context of library requirements. Trade-off decisions may include selecting open-source tools versus custom development, reusing existing components, or choosing between incremental implementation and one-time deployment. Prototyping is encouraged to evaluate technology support for library processes.

PLANNING PHASE

The Planning Phase is one of the most critical stages in the development of the Library Management System. Proper planning at this stage ensures that project activities are well coordinated and potential risks are identified and managed effectively. The level of detail and formality in planning is determined by the scope, complexity, and risks associated with the project. This phase builds upon the

information gathered during the initiation phase by clearly defining the tasks, resources, and timelines required to complete the system successfully.

During this phase, the Project Manager coordinates discussions among librarians, administrators, security personnel, designers, developers, and database managers. These interactions help identify and document detailed functional, security, and network requirements of the Library Management System. A structured project plan is developed that outlines the development approach, tools and technologies to be used, tasks to be performed, required resources, project schedule, and user involvement throughout the project lifecycle.

A comprehensive Project Management Plan is prepared during the Planning Phase. This plan includes components related to system development strategy, configuration management, quality assurance, system security, verification and validation, and overall systems engineering management. Personnel responsibilities, estimated costs, milestones, and target completion dates are clearly defined. Effective planning in this phase ensures that the Library Management System is developed in a controlled, efficient, and timely manner while meeting user and organizational requirements.

REQUIREMENTS ANALYSIS PHASE

The Requirements Analysis Phase formally defines the detailed functional and non-functional requirements of the Library Management System using the high-level requirements identified during the Initiation, System Concept Development, and Planning phases. This phase focuses on translating business needs into clear, precise, and complete system requirements that will guide system design and development.

During this phase, requirements are specified in terms of system functionality, data handling, performance, security, and maintainability. Each requirement is defined at a level of detail sufficient for the design phase to begin. All requirements are made measurable and testable to ensure they can be validated against the original business objectives identified in the Initiation Phase. Acceptance criteria for the

system are documented and later used to evaluate system performance through formal testing.

The purposes of the Requirements Analysis Phase are to:

- ✓ Further define and refine functional and data requirements related to book management, member management, issue–return processes, and reporting, and document them in a comprehensive Requirements Specification Document.
- ✓ Complete business process reengineering by analyzing how information flows within the library system, identifying who generates data, how it is processed, and where it is stored or utilized.
- ✓ Develop detailed data and process models, including system inputs, outputs, workflows, and internal processes required to support library operations.
- ✓ Develop test and evaluation requirements that will be used to measure system performance, accuracy, security, and reliability, and document them in the Test and Evaluation Master Plan.

This phase ensures that the Library Management System is clearly defined, aligned with organizational needs, and ready for systematic design and implementation.

DESIGN PHASE

The System Design Phase involves transforming the informational, functional, and network requirements identified during the Initiation, Planning, and Requirements Analysis phases into detailed and unified design specifications. These specifications act as a blueprint that developers follow during the implementation of the Library Management System. The objective of this phase is to design a system that fully satisfies user requirements while minimizing technical and operational risks.

During this phase, program designs are developed using structured design approaches. In a top-down approach, major system components such as book management, member management, and transaction processing modules are identified first, followed by detailed design of their subcomponents. In a bottom-up approach, smaller components are designed initially and then integrated into larger subsystems. Modern design practices also make use of prototyping tools to create

mock-ups of user interfaces, database structures, and system architecture. These prototypes are reviewed and refined through iterative feedback from end users, designers, developers, database administrators, and network personnel.

DEVELOPMENT PHASE

The Development Phase involves the actual construction of the Library Management System based on the approved System Design Document. In this phase, the system design is translated into working software components. Developers write source code, create database structures, and integrate different modules to build a fully functional system that meets the specified requirements.

During this phase, individual modules such as book management, member management, issue and return processing, authentication, and reporting are developed separately. Python is used to implement application logic, while MySQL is used to design and manage the backend database. Each module follows predefined coding standards to ensure consistency, readability, and maintainability. Configuration management practices are applied to track changes and control versions of the software.

TESTING AND IMPLEMENTATION PHASE

The Testing and Implementation Phase ensures that the Library Management System functions correctly and meets all specified requirements before being made available for actual use. This phase focuses on validating system performance, identifying defects, and confirming that the system is reliable, secure, and user-ready. Only after successful testing is the system implemented in the real operating environment.

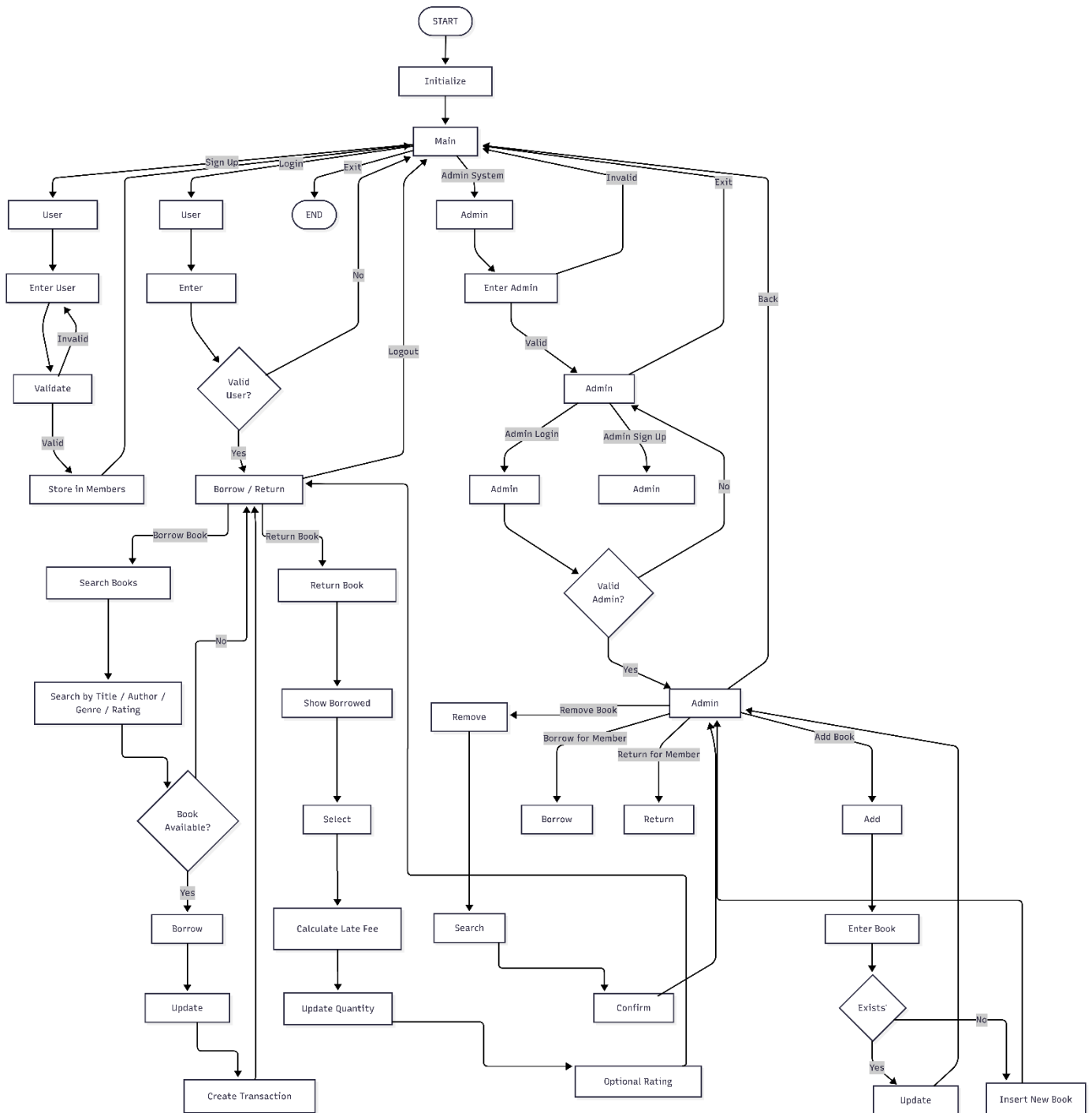
During the testing phase, the system undergoes various levels of testing to verify its functionality and stability. Unit testing is performed to check individual modules such as book management and member login. Integration testing ensures proper interaction between modules, while system testing evaluates the complete system as a whole. User Acceptance Testing is conducted to confirm that the system satisfies user requirements and business objectives. Any identified errors are corrected before implementation.

OPERATIONS AND MAINTENANCE PHASE

The Operations and Maintenance Phase begins once the Library Management System has been successfully tested and implemented. This phase focuses on the day-to-day operation of the system and ensures that it continues to function effectively in the live environment. The system is monitored regularly to maintain performance, reliability, and security while supporting ongoing library activities. During operations, administrators and users utilize the system for routine tasks such as managing books, handling member records, and processing issue and return transactions. System performance is continuously observed to detect errors, system failures, or security vulnerabilities. Any issues identified during regular use are documented and addressed promptly to avoid disruption of library services.

CHAPTER 5

FLOW CHART



CHAPTER 6

SOURCE CODE

1. main.py :

```
from users import login, signup
from library_administration import admin_menu
from db_config import db_setup
import os

def main( ):
    db_setup( )
    while True:
        try:
            print("\n=== Library Management System ===")
            print("\n1. Login")
            print("2. Sign Up")
            print("3. Library Administration System")
            print("4. Exit")
            print("\n=====")
            choice = input("\nSelect an option (1-4): ").strip( )
            if choice == '1':
                os.system('cls')
                login( )
            elif choice == '2':
                os.system('cls')
                signup( )
            elif choice == '3':
                os.system('cls')
                admin_menu( )
            elif choice == '4':
                os.system('cls')
                print("Goodbye!")
                break
            else:
                print("Invalid choice. Try again.")
                input("Press Enter to continue...")
                os.system('cls')
                continue
        except:
            os.system('cls')
            continue
    main( )
```

2. db_config.py :

```
import mysql.connector
import os
```

```

def connect_db( ):
    return mysql.connector.connect(
        host="localhost",
        user="root",
        password="root",
        database="library_management"
    )
def db_setup( ):
    try:
        conn = mysql.connector.connect(
            host="localhost",
            user="root",
            password="2009")
        cursor = conn.cursor( )
        cursor.execute("SHOW DATABASES LIKE 'library_management'")
        exists = cursor.fetchone( )
        if not exists:
            print("Setting up database for the first time...")
            with open("schema.sql", "r") as f:
                sql_commands = f.read( )
            for command in sql_commands.split(';'):
                cmd = command.strip( )
                if cmd:
                    cursor.execute(cmd)
            conn.commit( )
            print("Database initialized successfully.\n")
            input("Press Enter to Initalize the Library Management System...")
            os.system('cls')
        except Exception as e:
            print(f" Database initialization failed: {e}")
        conn.close( )

```

3. users.py :

```

from db_config import connect_db
from datetime import date
from book import books
import os
def signup( ):
    print("\n===== Sign up =====\n")
    while True:
        user = input("Enter username: ").strip( )
        if not user :
            print("Error: Username is required.")
            input("Press Enter to continue...")
            os.system('cls')

```

```

    continue
if len(user) < 5:
    print("Error: Username must be at least 5 characters long.")
    input("Press Enter to continue...")
    os.system('cls')
    continue
break
while True:
    pwd = input("Enter password: ").strip()
    if not pwd:
        print("Error: Password is required.")
        input("Press Enter to continue...")
        os.system('cls')
        continue
    if len(pwd) < 8:
        print("Error: Password must be at least 8 characters long.")
        input("Press Enter to continue...")
        os.system('cls')
        continue
    break
confirm_pwd = input("Confirm password: ").strip()
if pwd != confirm_pwd:
    print("Error: Passwords do not match.")
    input("Press Enter to return...")
    os.system('cls')
    return
while True:
    full_name = input("Enter your full name: ").strip()
    if not full_name :
        print("Error: Full Name is required.")
        input("Press Enter to continue...")
        os.system('cls')
        continue
    if len(full_name) < 5:
        print("Error: Full Name must be at least 5 characters long.")
        input("Press Enter to continue...")
        os.system('cls')
        continue
    break
while True:
    email = input("Enter your email: ").strip()
    if not email :
        print("Error: Email is required.")
        input("Press Enter to continue...")

```

```

        os.system('cls')
        continue
    if not (email.endswith("@gmail.com") or email.endswith("@outlook.com")):
        print("Error: Use a Gmail account or Outlook account to sign up.")
        input("Press Enter to continue...")
        os.system('cls')
        continue
    break
while True:
    phone = input("Enter your phone number: ").strip()
    if not phone :
        print("Error: Phone Number is required.")
        input("Press Enter to continue...")
        os.system('cls')
        continue
    if not(phone.isdigit() or len(phone)!=10):
        print("Error: Enter a valid phone Number")
        input("Press Enter to continue...")
        os.system('cls')
        continue
    break
while True:
    address = input("Enter your address: ").strip()
    if not address :
        print("Error: Address is required.")
        input("Press Enter to continue...")
        os.system('cls')
        continue
    break
conn = connect_db()
cursor = conn.cursor()
try:
    cursor.execute("INSERT INTO members (username, password, full_name,
email, phone, address, joined_date) VALUES (%s, %s, %s, %s, %s, %s, %s)", (user,
pwd, full_name, email, phone, address , date.today( )))
    conn.commit()
    print("\n=====")
    os.system('cls')
    print("Sign Up Success: Account created.")
except :
    print("Error: Could not create account. Try a different username.")
    input("Press Enter to return...")
    os.system('cls')
    return

```

```

    conn.close()
def login():
    print("\n===== Login =====\n")
    user = input("Enter username: ").strip()
    pwd = input("Enter password: ").strip()
    if not user or not pwd:
        print("Error: Both fields are required.")
        input("Press Enter to return...")
        os.system('cls')
        return
    conn = connect_db()
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM members WHERE username=%s AND
password=%s", (user, pwd))
    result = cursor.fetchone()
    conn.close()
    print("\n=====")
    os.system('cls')
    if result:
        print(f"\nLogin Success: Welcome, {user}!")
        books(result[0])
    else:
        print("Login Failed: Invalid username or password.")
        input("Press Enter to return...")
        os.system('cls')
        return

```

4. library_administration.py :

```

from db_config import connect_db
from utilies import add_book, remove_book, borrow_book, return_book
import os
def admin_menu():
    print("Enter Admin Access Password to Proceed...")# Admin code: admin_root
    admin_code = input("").strip()
    if admin_code != "admin_root":
        print("\nAdmin Access DENIED")
        input("Press Enter to return to the main menu...")
        os.system('cls')
        return
    os.system('cls')
    print("Welcome to the Library Administration System!")
    while True:
        print("\n===== Admin Access =====\n")
        print("1. Admin Login")
        print("2. Admin Sign Up")

```

```

print("3. Exit")
print("\n===== \n")
choice = input("Enter choice: ").strip()
if choice == '1':
    os.system('cls')
    admin_login()
elif choice == '2':
    os.system('cls')
    admin_signup()
elif choice == '3':
    os.system('cls')
    break
else:
    print("Invalid choice. Try again.")
    input("Press Enter to continue...")
    os.system('cls')
    os.system('cls')
def admin_signup():
    from db_config import connect_db
    from datetime import date
    conn = connect_db()
    cursor = conn.cursor()
    while True:
        print("\n===== Admin Sign Up ===== \n")
        print("Please enter the following details to create an admin account.")
        admin_user = input("Enter username: ").strip()
        if not admin_user:
            print("Error: Username is required.")
            input("Press Enter to continue...")
            os.system('cls')
            continue
        if len(admin_user) < 5:
            print("Error: Username must be at least 5 characters long.")
            input("Press Enter to continue...")
            os.system('cls')
            continue
        break
    while True:
        admin_pwd = input("Enter password: ").strip()
        if not admin_pwd:
            print("Error: Password is required.")
            input("Press Enter to continue...")
            os.system('cls')
            continue

```

```

if len(admin_pwd) < 8:
    print("Error: Password must be at least 8 characters long.")
    input("Press Enter to continue...")
    os.system('cls')
    continue
admin_confirm_pwd = input("Confirm password: ").strip( )
if admin_pwd != admin_confirm_pwd:
    print("Error: Passwords do not match.")
    input("Press Enter to continue...")
    os.system('cls')
    continue
break
try:
    cursor.execute("""
        INSERT INTO administration (username, password, created_on)
        VALUES (%s, %s, %s)
        """, (admin_user, admin_pwd, date.today( )))
    conn.commit( )
except:
    print(" Error: That username may already exist.")
    input("Press Enter to continue...")
    os.system('cls')
conn.close( )
print("\n===== \n")
input("Press Enter to return to the admin menu...")
os.system('cls')
print(f"Administrator {admin_user} Sign Up Successful!")
def admin_login( ):
    conn = connect_db( )
    cursor = conn.cursor( )
    while True:
        print("\n===== Admin Login =====")
        print("Please enter your admin credentials to access the library
administration system.\n")
        admin_user = input("Admin Username: ").strip( )
        admin_pwd = input("Admin Password: ").strip( )
        try:
            cursor.execute("""SELECT * FROM administration
            WHERE username = %s AND password = %s""",(admin_user, admin_pwd))
        except:
            input("Invalid admin credentials. Press Enter to try again...")
            os.system('cls')
            continue
        result = cursor.fetchone( )

```

```

if result:
    os.system('cls')
    print(f"\nWelcome, Administrator {admin_user}!")
    admin_panel(admin_user, admin_pwd)
    conn.close()
    print("\n=====")
    break
else:
    print("Invalid admin credentials.")
    input("Press Enter to continue...")
    os.system('cls')
    continue
conn.close()
def admin_panel(user,pwd):
    conn=connect_db()
    cursor=conn.cursor()
    while True:
        print("\n===== Admin Panel =====\n")
        print("1. Add a Book")
        print("2. Remove a Book")
        print("3. Borrow a Book")
        print("4. Return a Book")
        print("5. Back to Main Menu")
        print("\n=====")
        choice = input("Enter your choice: ").strip()
        try:
            if choice == '1':
                os.system('cls')
                add_book()
            elif choice == '2':
                os.system('cls')
                remove_book()
            elif choice == '3':
                os.system('cls')
                member_username = input("Enter Member Username to borrow for: ")
                member_username = member_username.strip()
                cursor.execute("SELECT id FROM members WHERE username = %s",
                    (member_username,))
                member = cursor.fetchone()
                if member:
                    borrow_book(member[0])
                else:
                    print("Member not found.")
                    input("Press Enter to return...")

```

```

elif choice == '4':
    os.system('cls')
    member_username = input("Enter Member Username to return for:
").strip( )
    cursor.execute("SELECT id FROM members WHERE username = %s",
(member_username,))
    member = cursor.fetchone( )
    if member:
        return_book(member[0])
    else:
        print("Member not found.")
        input("Press Enter to return...")
elif choice == '5':
    return
except Exception as e:
    print(f"An error occurred: {e}")
    input("Press Enter to continue...")
    continue

```

5. utilies.py :

```

from datetime import date, timedelta
from db_config import connect_db
import os

```

```

def borrow_book(member_id):
    conn = connect_db( )
    cursor = conn.cursor( )
    cursor.execute("SELECT * FROM books WHERE quantity > 0")
    available_books = cursor.fetchall( )
    if not available_books:
        print("No books available for borrowing!")
        print("\n=====")
        return
    while True:
        print("\n===== Borrow a Book =====\n")
        print("1. Search by Title")
        print("2. Search by Author")
        print("3. Search by Genre")
        print("4. Search by Rating")
        print("5. Show all the books in Library")
        print("6. Back to Borrow/Return System")
        print("\n=====")
        print( )
        search_choice = input("Enter your choice (1-6): ").strip( )
        os.system('cls')

```

```

if search_choice == "1":
    title = input("Enter the title of the book: ").strip( )
    book_update("title", title, member_id)
    input("Press Enter to return...")
    os.system('cls')
elif search_choice == "2":
    author = input("Enter the author of the book: ").strip( )
    book_update("author", author, member_id)
    input("Press Enter to return...")
    os.system('cls')
elif search_choice == "3":
    print('Available Genres:')
    print( )
    cursor.execute("SELECT DISTINCT genre FROM books WHERE quantity >
0;")
    genres_available = cursor.fetchall( )
    for genres in genres_available:
        print(str(genres_available.index(genres)+1)+'.'+ genres[0])
    genre_choice = input('\nWould you like to select from the above genres (
Y/N )?').strip( ).lower( )
    if genre_choice != 'y':
        os.system('cls')
        continue
    while True:
        try:
            genre_index = int(input("Enter the No. of the genre of the book: ").strip(
))
            if genre_index < 1 or genre_index > len(genres_available):
                print("Invalid genre index. Please try again.")
                input("Press Enter to continue...")
                os.system('cls')
                continue
            break
        except ValueError:
            print("Invalid input. Please enter a number.")
            input("Press Enter to continue...")
            os.system('cls')
            continue
    genre = genres_available[genre_index-1][0]
    book_update("genre", genre, member_id)
    input("Press Enter to return...")
    os.system('cls')
elif search_choice == "4":
    print('Rating of Books lies between 1 to 5')

```

```

try:
    rating_input = input("Enter the rating of the book: ").strip( )
    if not rating_input.isdigit( ):
        print("Invalid input. Please enter a number.")
        input("Press Enter to continue...")
        os.system('cls')
        continue
    rating = int(rating_input)
    if rating < 1 or rating > 5:
        print("Rating must be between 1 and 5.")
        input("Press Enter to continue...")
        os.system('cls')
        continue
    book_update("rating", rating, member_id)
    input("Press Enter to return...")
    os.system('cls')
except ValueError:
    print("Invalid input. Please enter a number for rating.").strip( )
    input("Press Enter to continue...")
    os.system('cls')
    continue
elif search_choice == "5":
    print('Available books:')
    for book in available_books:
        print(available_books.index(book)+1, '. ', book[1])
    print("\n=====\\n")
    print("Do You want to borrow a book from the above list?")
    sub_desire = input("Enter 'Y' or 'N': ").strip( ).lower( )
    if sub_desire != 'y':
        os.system('cls')
        continue
    print( )
    try:
        sno_choice = input("Enter the serial numbers of the books you want to
borrow separated by commas: ").strip( )
        sno_choice = [int(i.strip( )) - 1 for i in sno_choice.split(",")]
    except ValueError:
        print("Invalid input. Please enter numbers separated by commas.")
        input("Press Enter to continue...")
        os.system('cls')
        return
    for sub_choice_index in sno_choice:
        if sub_choice_index < 0 or sub_choice_index >= len(available_books):
            os.system('cls')

```

```

        print(f"Invalid choice. {sub_choice_index + 1} can't exist. Please enter
a valid serial number.")
        input("Press Enter to continue...")
        os.system('cls')
        continue
        title = available_books[sub_choice_index][1]
        author = available_books[sub_choice_index][2]
        os.system('cls')
        print(f"\nSelected Book:\nTitle: {title}\nAuthor: {author}")
        sub_result = available_books[sub_choice_index]
        book_fees(member_id, sub_result, title)
        print("\n=====")
        input("Press Enter to return...")
        os.system('cls')
        input("Press Enter to return...")
        os.system('cls')
    elif search_choice == "6":
        break
    else:
        print("Invalid choice. Please enter 1, 2, 3, 4, 5 or 6.")
        input("Press Enter to continue...")
        os.system('cls')
        continue
    conn.close()
def book_update(feature_name, feature, member_id):
    conn = connect_db()
    cursor = conn.cursor()
    if feature_name in ['title', 'author']:
        feature = feature.lower()
        cursor.execute(f"SELECT * FROM books WHERE LOWER({feature_name}) =
%s AND quantity > 0", (feature,))
    elif feature_name in ['rating', 'genre']:
        cursor.execute(f"SELECT * FROM books WHERE {feature_name} = %s AND
quantity > 0", (feature,))
    result = cursor.fetchall()
    os.system('cls')
    if result and len(result) == 1:
        for book in result:
            print('\n',f"Title: {book[1]}\n Author: {book[2]}\n Genre: {book[3]}\n
Quantity: {book[4]}\n Rating: {book[5]}\n")
            print("\n=====")
            print("Do You want to borrow a book from the above list?")
            sub_desire = input("Enter 'Y' or 'N': ").strip().lower()
            if sub_desire != 'y':

```

```

        os.system('cls')
        return
    sub_choice_index = 0
    title = result[sub_choice_index][1]
    author = result[sub_choice_index][2]
    os.system('cls')
    print(f"\nSelected Book:\nTitle: {title}\nAuthor: {author}")
    sub_result = result[sub_choice_index]
    book_fees(member_id, sub_result, title)
    print("\n=====")
elif result:
    print( )
    for book in result:
        print(result.index(book)+1, ". \n", f"Title: {book[1]}\n Author: {book[2]}\n
Genre: {book[3]}\n Quantity: {book[4]}\n Rating: {book[5]}\n")
        print("\n=====")
        print("Do You want to borrow a book from the above list?")
        sub_desire = input("Enter 'Y' or 'N': ").strip( ).lower( )
        if sub_desire != 'y':
            os.system('cls')
            return
        print( )
    try:
        sno_choice = input("Enter the serial numbers of the books you want to
borrow separated by commas: ").strip( )
        sno_choice = [int(i.strip( )) - 1 for i in sno_choice.split(",")]
    except ValueError:
        print("Invalid input. Please enter numbers separated by commas.")
        input("Press Enter to continue...")
        os.system('cls')
        return
    for sub_choice_index in sno_choice:
        if sub_choice_index < 0 or sub_choice_index >= len(result):
            print(f"Invalid choice. {sub_choice_index + 1} can't exist. Please enter a
valid serial number.")
            input("Press Enter to continue...")
            os.system('cls')
            continue
        title = result[sub_choice_index][1]
        author = result[sub_choice_index][2]
        os.system('cls')
        print(f"\nSelected Book:\nTitle: {title}\nAuthor: {author}")
        sub_result = result[sub_choice_index]
        book_fees(member_id, sub_result, title)

```

```

        print("\n=====\\n")
        input("Press Enter to return...")
        os.system('cls')
    else:
        print('Book is not Available.')
        input("Press Enter to return...")
        os.system('cls')
        return
    conn.close( )
def book_fees(member_id, sub_result, title):
    conn = connect_db( )
    cursor = conn.cursor( )
    due_date = date.today( ) + timedelta(days=60)
    fee = 0.00
    print(f"\n{title} is expected to be returned on {due_date}")
    print("A fee of ₹10 per day will be charged after 60 days.")
    book_id = sub_result[0]
    cursor.execute("UPDATE books SET quantity = quantity - 1 WHERE book_id =
%s AND quantity > 0", (book_id,))
    if cursor.rowcount == 0:
        print(f"\nError: '{title}' is currently out of stock.")
        conn.rollback( )
        conn.close( )
        return
    cursor.execute("""
INSERT INTO transactions (book_id, member_id, borrow_date, expected_date,
fees)
VALUES (%s, %s, %s, %s, %s)
""", (book_id, member_id, date.today( ), due_date, fee))
    conn.commit( )
    conn.close( )
    print(f"{title} borrowed successfully!")
def return_book(member_id):
    conn = connect_db( )
    cursor = conn.cursor( )
    print("\n===== Return a book =====\\n")
    print("Returnable Books:")
    cursor.execute("""
SELECT t.transaction_id, t.expected_date, t.fees, b.book_id, b.title
FROM transactions t
JOIN books b ON t.book_id = b.book_id
WHERE t.member_id = %s
AND t.return_date IS NULL;
""", (member_id,))

```

```

rows = cursor.fetchall( )
if not rows:
    print("No books to return.")
    print("\n=====")
    input("Press Enter to return..")
    os.system('cls')
    conn.close( )
    return
for i, row in enumerate(rows):
    print(f'{i + 1}. {row[4]} (Due: {row[1]})')
print("\n=====\\n")
try:
    selection = int(input("Enter the number of the book to return: ").strip( ))
    if selection < 1 or selection > len(rows):
        print("Invalid selection.")
        conn.close( )
        input("Press Enter to return...")
        return
    transaction = rows[selection - 1]
    transaction_id = transaction[0]
    expected_date = transaction[1]
    initial_fees = transaction[2]
    book_id = transaction[3]
    book_title = transaction[4]
except ValueError:
    print("Invalid input. Please enter a number.")
    conn.close( )
    input("Press Enter to return...")
    os.system('cls')
    return
days_late = (date.today( ) - expected_date).days
late_fee_per_day = 10
if days_late <= 0:
    late_fee = 0.0
else:
    late_fee = days_late * late_fee_per_day
total_fee = float(initial_fees) + late_fee
print(f"Return the book '{book_title}' and Pay ₹{total_fee}!")
input("Press Enter to Pay with Cash..")
os.system('cls')
cursor.execute("UPDATE books SET quantity = quantity + 1 WHERE book_id =
%s", (book_id,))
rate_choice = input("Would you like to rate the book? ( Y/N ):").strip( ).lower(
)

```

```

rating = None
if rate_choice == "y":
    try:
        rating = int(input("Rate the book (1 to 5): ").strip( ))
        if rating < 1 or rating > 5:
            print("Rating out of range. Skipping rating.")
            rating = None
    except ValueError :
        print("Invalid rating. Skipping rating.")
        rating = None
if rating is None:
    print("We respect your valuable time! Feedback is optional.")
    cursor.execute("""
        UPDATE transactions
        SET return_date = %s
        WHERE transaction_id = %s
        """, (date.today( ), transaction_id))
    os.system('cls')
else:
    cursor.execute("""
        UPDATE transactions
        SET return_date = %s, rating = %s
        WHERE transaction_id = %s
        """, (date.today( ), rating, transaction_id))
if rating is not None:
    cursor.execute("""
        SELECT AVG(rating)
        FROM transactions
        WHERE book_id = %s AND rating IS NOT NULL
        """, (book_id,))
    result = cursor.fetchone( )
    if result and result[0] is not None:
        avg_rating = round(result[0], 2)
        cursor.execute("UPDATE books SET rating = %s WHERE book_id = %s",
        (avg_rating, book_id))
        print(f"Thank you for rating! The new average rating for this book is
        {avg_rating}.")
    print("Book returned successfully!")
    print("\n=====")
    input("Press Enter to return..")
    os.system('cls')
    conn.commit( )
    conn.close( )
def add_book( ):

```

```

conn = connect_db( )
cursor = conn.cursor( )
print("===== Add a Book =====\n")
print("Please enter the book details below:")
while True:
    title = input("Enter Book Title: ").strip( ).capitalize( )
    if title :
        break
while True:
    author = input("Enter Book Author: ").strip( ).capitalize( )
    if author:
        break
while True:
    genre = input("Enter Book Genre: ").strip( ).capitalize( )
    if genre:
        break
while True:
    try:
        quantity = int(input("Quantity: ").strip( ))
        if quantity <= 0:
            print('Quantity can't be less than 1.')
            break
    except ValueError:
        print("Invalid quantity. Enter a valid quantity.")
        input("Press Enter to continue...")
        os.system('cls')
        continue
while True:
    invoice_no = input("Enter Invoice Number: ").strip( )
    if invoice_no:
        break
while True:
    publisher_name = input("Enter Publisher Name: ").strip( )
    if publisher_name:
        break
while True:
    store_name = input("Enter Store Name: ").strip( )
    if store_name:
        break
while True:
    try:
        amt = float(input("Enter Price: "))
        if amt >= 0:
            break

```

```

except ValueError:
    print("Enter a valid Price.")
    input("Press Enter to continue...")
    os.system('cls')
    continue
while True:
    purchase_date_str = input("Enter Purchase Date (YYYY-MM-DD): ").strip()
    if not purchase_date_str:
        print("Purchase date is required.")
        input("Press Enter to continue...")
        os.system('cls')
        continue
    try:
        year, month, day = map(int, purchase_date_str.replace('/', '-').split('-'))
        purchase_date = date(year, month, day)
        if purchase_date > date.today():
            print("Purchase date cannot be in the future.")
            input("Press Enter to continue...")
            os.system('cls')
            continue
        break
    except ValueError:
        print("Invalid date format or value. Please use YYYY-MM-DD.")
        input("Press Enter to continue...")
        os.system('cls')
        continue
cursor.execute("""
SELECT book_id
FROM books
WHERE LOWER(title) = %s
AND LOWER(author) = %s
AND LOWER(genre) = %s
""", (title.lower(), author.lower(), genre.lower()))
existing = cursor.fetchone()
if existing:
    cursor.execute("""
UPDATE books
SET quantity = quantity + %s,
    invoice_no = %s,
    purchase_date = %s,
    publisher_name = %s,
    store_name = %s,
    amt = %s
WHERE book_id = %s

```

```

        """, (quantity, invoice_no, purchase_date, publisher_name, store_name, amt,
existing[0]))
    print("Book already exists. Updated quantity and details.")
else:
    cursor.execute("""
        INSERT INTO books (title, author, genre, quantity, invoice_no,
purchase_date, publisher_name, store_name, amt)
        VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s)
        """, (title, author, genre, quantity, invoice_no, purchase_date, publisher_name,
store_name, amt))
    print("New book added successfully.")
    conn.commit()
    conn.close()
    print("Book added successfully!")
    print("\n=====")
    input("Press Enter to return...")
    os.system('cls')
def remove_book():
    conn = connect_db()
    cursor = conn.cursor()
    print("\n===== Remove Book =====\n")
    while True:
        title = input("Enter the title of the book: ").strip()
        if title:
            break
    while True:
        author = input("Enter the author of the book: ").strip()
        if author:
            break
    cursor.execute("SELECT * FROM books WHERE title = %s AND author = %s",
(title, author))
    results = cursor.fetchall()
    if not results:
        print(" Book not found.")
        conn.close()
        input("Press Enter to Return...")
        os.system('cls')
        return
    print(f"\nFound {len(results)} book(s):")
    for i, book in enumerate(results):
        print(f"{i + 1}. Title: {book[1]}, Author: {book[2]}, Genre: {book[3]},
Publisher: {book[9]}")
    print("\n=====")
    try:

```

```

selection = int(input("Enter the number of the book to delete: ").strip( ))
if selection < 1 or selection > len(results):
    print("Invalid selection.")
    conn.close( )
    input("Press Enter to return...")
    return
selected_book = results[selection - 1]
book_id = selected_book[0]
confirm = input(f'Are you sure you want to delete '{selected_book[1]}' (ID:
{book_id})? ( Y/N ): ').strip( ).lower( )
if confirm == 'y':
    cursor.execute("DELETE FROM books WHERE book_id = %s", (book_id,))
    conn.commit( )
    print("Book removed successfully.")
else:
    print("Operation cancelled.")
except ValueError:
    print("Invalid input. Please enter a number.")
    input("Press Enter to continue...")
    os.system('cls')
print("\n=====")
input("Press Enter to return...")
os.system('cls')
conn.close( )

```

6. schema.sql :

```

CREATE DATABASE IF NOT EXISTS library_management;
USE library_management;
CREATE TABLE administration (
    admin_id INT NOT NULL AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(255) NOT NULL UNIQUE,
    password VARCHAR(255) NOT NULL,
    created_on DATETIME DEFAULT CURRENT_TIMESTAMP
);
INSERT INTO administration (username, password)
VALUES ('Admin', '1234');
CREATE TABLE books (
    book_id INT NOT NULL AUTO_INCREMENT PRIMARY KEY,
    title VARCHAR(100) NOT NULL,
    author VARCHAR(50) NOT NULL,
    genre VARCHAR(50),
    quantity INT DEFAULT 0,
    rating DECIMAL(3,2),
    purchase_date DATE ,
    invoice_no VARCHAR(20) NOT NULL DEFAULT 'UNKNOWN',

```

```

    publisher_name VARCHAR(100) NOT NULL DEFAULT 'UNKNOWN',
    store_name VARCHAR(100) NOT NULL DEFAULT 'UNKNOWN',
    amt DECIMAL(10,2) NOT NULL DEFAULT 0.00
);
INSERT INTO books (title, author, genre, quantity, rating, purchase_date)
VALUES
('To Kill a Mockingbird', 'Harper Lee', 'Fiction', 1, 4, '2025-05-10'),
('1984', 'George Orwell', 'Dystopian', 4, 5, '2025-05-10'),
('Pride and Prejudice', 'Jane Austen', 'Romance', 3, 3, '2025-05-10'),
('The Great Gatsby', 'F. Scott Fitzgerald', 'Classic', -7, 4, '2025-05-10'),
('Moby Dick', 'Herman Melville', 'Adventure', 1, 2, '2025-05-10'),
('The Hobbit', 'J.R.R. Tolkien', 'Fantasy', 5, 5, '2025-05-10'),
('War and Peace', 'Leo Tolstoy', 'Historical Fiction', 2, 3, '2025-05-10'),
('Jane Eyre', 'Charlotte Brontë', 'Romance', 2, 4, '2025-05-10'),
('The Catcher in the Rye', 'J.D. Salinger', 'Coming-of-Age', 3, 2, '2025-05-10'),
('The Lord of the Rings', 'J.R.R. Tolkien', 'Fantasy', 3, 5, '2025-05-10');
CREATE TABLE members (
    id INT NOT NULL AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(50) NOT NULL UNIQUE,
    password VARCHAR(255) NOT NULL,
    full_name VARCHAR(100),
    email VARCHAR(100),
    phone VARCHAR(20),
    address VARCHAR(255),
    joined_date DATE
);
INSERT INTO members (username, password, full_name, email, phone, address,
joined_date)
VALUES ('User', '1234', 'UserInterface', 'userinterface@gmail.com',
'9870645321', '7, Park Avenue.', '2025-05-02');
CREATE TABLE transactions (
    transaction_id INT NOT NULL AUTO_INCREMENT PRIMARY KEY,
    book_id INT,
    member_id INT,
    borrow_date DATE,
    return_date DATE,
    expected_date DATE,
    rating INT,
    fees DECIMAL(10,2) DEFAULT 0.00,
    FOREIGN KEY (book_id) REFERENCES books(book_id) ON DELETE CASCADE,
    FOREIGN KEY (member_id) REFERENCES members(id) ON DELETE CASCADE
);
INSERT INTO transactions (book_id, member_id, borrow_date, return_date,
expected_date, rating, fees)

```

VALUES

```
(1, 1, '2025-05-01', '2025-05-10', '2025-05-08', 4, 0.00),
(2, 1, '2025-05-01', NULL, '2025-06-30', NULL, 0.00),
(3, 1, '2025-05-02', '2025-05-25', '2025-06-30', 3, 0.00),
(4, 1, '2025-05-03', NULL, '2025-07-01', NULL, 0.00),
(5, 1, '2025-05-04', '2025-05-20', '2025-07-03', 2, 0.00),
(6, 1, '2025-05-05', NULL, '2025-07-04', NULL, 0.00),
(7, 1, '2025-05-06', NULL, '2025-07-05', NULL, 0.00),
(8, 1, '2025-05-07', '2025-06-07', '2025-07-06', 4, 0.00),
(9, 1, '2025-05-08', NULL, '2025-07-07', NULL, 0.00),
(10, 1, '2025-05-09', '2025-06-15', '2025-07-08', 5, 0.00);
```

7. book.py :

```
from utilies import borrow_book, return_book
import os
def books(member_id):
    while True:
        print("\n==== Borrow/Return System =====\n")
        print("1. Borrow a Book")
        print("2. Return a Book")
        print("3. Logout")
        print("\n=====")
        choice = input("Enter your choice (1-3): ").strip()
        if choice == "1":
            os.system('cls')
            borrow_book(member_id)
        elif choice == "2":
            os.system('cls')
            return_book(member_id)
        elif choice == "3":
            print("Logging out...")
            os.system('cls')
            break
        else:
            print("Invalid choice. Please enter 1, 2, or 3.")
            continue
```

CHAPTER 7

OUTPUT

```
Setting up database for the first time...
Database initialized successfully.
Press Enter to Initalize the Library Management System...
=== Library Management System ===
1. Login
2. Sign Up
3. Library Administration System
4. Exit
=====
Select an option (1-4): 2
===== Sign up =====
Enter username: Nolan
Enter password: 12345678
Confirm password: 12345678
Enter your full name: Nolan Rift
Enter your email: nolanrift2003@gmail.com
Enter your phone number: 9876456411
Enter your address: 7, Park Avenue, Ironcrest District
Sign Up Success: Account created.
=== Library Management System ===
1. Login
2. Sign Up
3. Library Administration System
4. Exit
=====
Select an option (1-4): 1
===== Login =====
Enter username: Nolan
Enter password: 12345678
Login Success: Welcome, Nolan!
===== Borrow/Return System =====
1. Borrow a Book
2. Return a Book
3. Logout
=====
Enter your choice (1-3): 1
===== Borrow a Book =====
1. Search by Title
2. Search by Author
3. Search by Genre
4. Search by Rating
```

5. Show all the books in Library
6. Back to Borrow/Return System

=====

Enter your choice (1-6): 1

Enter the title of the book: 1984

Title: 1984

Author: George Orwell

Genre: Dystopian

Quantity: 3

Rating: 5.00

=====

Do You want to borrow a book from the above list?

Enter 'Y' or 'N': Y

Selected Book:

Title: 1984

Author: George Orwell

1984 is expected to be returned on 2026-02-01

A fee of ₹10 per day will be charged after 60 days.

1984 borrowed successfully!

=====

Press Enter to return...

===== Borrow a Book =====

1. Search by Title

2. Search by Author

3. Search by Genre

4. Search by Rating

5. Show all the books in Library

6. Back to Borrow/Return System

=====

Enter your choice (1-6): 2

Enter the author of the book: George Orwell

Title: 1984

Author: George Orwell

Genre: Dystopian

Quantity: 2

Rating: 5.00

=====

Do You want to borrow a book from the above list?

Enter 'Y' or 'N': Y

Selected Book:

Title: 1984

Author: George Orwell

1984 is expected to be returned on 2026-02-01

A fee of ₹10 per day will be charged after 60 days.

1984 borrowed successfully!

=====

Press Enter to return...

===== Borrow a Book =====

1. Search by Title
2. Search by Author
3. Search by Genre
4. Search by Rating
5. Show all the books in Library
6. Back to Borrow/Return System

=====

Enter your choice (1-6): 3

Available Genres:

- 1.Fiction
- 2.Dystopian
- 3.Romance
- 4.Adventure
- 5.Fantasy
- 6.Historical Fiction
- 7.Coming-of-Age

Would you like to select from the above genres (Y/N)?Y

Enter the No. of the genre of the book: 4

Title: Moby Dick

Author: Herman Melville

Genre: Adventure

Quantity: 1

Rating: 2.00

=====

Do You want to borrow a book from the above list?

Enter 'Y' or 'N': Y

Selected Book:

Title: Moby Dick

Author: Herman Melville

Moby Dick is expected to be returned on 2026-02-01

A fee of ₹10 per day will be charged after 60 days.

Moby Dick borrowed successfully!

=====

Press Enter to return...

===== Borrow a Book =====

1. Search by Title
2. Search by Author
3. Search by Genre
4. Search by Rating
5. Show all the books in Library

6. Back to Borrow/Return System

=====

Enter your choice (1-6): 4

Rating of Books lies between 1 to 5

Enter the rating of the book: 4

1 .

Title: To Kill a Mockingbird

Author: Harper Lee

Genre: Fiction

Quantity: 1

Rating: 4.00

2 .

Title: Jane Eyre

Author: Charlotte Brontë

Genre: Romance

Quantity: 2

Rating: 4.00

=====

Do You want to borrow a book from the above list?

Enter 'Y' or 'N': Y

Enter the serial numbers of the books you want to borrow separated by commas:

1,2

Selected Book:

Title: To Kill a Mockingbird

Author: Harper Lee

To Kill a Mockingbird is expected to be returned on 2026-02-01

A fee of ₹10 per day will be charged after 60 days.

To Kill a Mockingbird borrowed successfully!

=====

Press Enter to return...

Selected Book:

Title: Jane Eyre

Author: Charlotte Brontë

Jane Eyre is expected to be returned on 2026-02-01

A fee of ₹10 per day will be charged after 60 days.

Jane Eyre borrowed successfully!

=====

Press Enter to return...

Press Enter to return...

===== Borrow a Book =====

1. Search by Title

2. Search by Author

3. Search by Genre

4. Search by Rating

5. Show all the books in Library
6. Back to Borrow/Return System

=====

Enter your choice (1-6): 5

Available books:

- 1 . To Kill a Mockingbird
- 2 . 1984
- 3 . Pride and Prejudice
- 4 . Moby Dick
- 5 . The Hobbit
- 6 . War and Peace
- 7 . Jane Eyre
- 8 . The Catcher in the Rye
- 9 . The Lord of the Rings

=====

Do You want to borrow a book from the above list?

Enter 'Y' or 'N': n

===== Borrow a Book =====

1. Search by Title
2. Search by Author
3. Search by Genre
4. Search by Rating
5. Show all the books in Library
6. Back to Borrow/Return System

=====

Enter your choice (1-6): 6

===== Borrow/Return System =====

1. Borrow a Book
2. Return a Book
3. Logout

=====

Enter your choice (1-3): 2

===== Return a book =====

Returnable Books:

1. 1984 (Due: 2026-02-01)
2. 1984 (Due: 2026-02-01)
3. Moby Dick (Due: 2026-02-01)
4. To Kill a Mockingbird (Due: 2026-02-01)
5. Jane Eyre (Due: 2026-02-01)

=====

Enter the number of the book to return: 1

Return the book '1984' and Pay ₹0.0!

Press Enter to Pay with Cash..

Would you like to rate the book? (Y/N):Y

Rate the book (1 to 5): 4

Thank you for rating! The new average rating for the book is 4.00.

Book returned successfully!

=====

Press Enter to return...

===== Borrow/Return System =====

1. Borrow a Book

2. Return a Book

3. Logout

=====

Enter your choice (1-3): 3

=== Library Management System ===

1. Login

2. Sign Up

3. Library Administration System

4. Exit

=====

Select an option (1-4): 3

Enter Admin Access Password to Proceed...

admin_root

Welcome to the Library Administration System!

===== Admin Access =====

1. Admin Login

2. Admin Sign Up

3. Exit

=====

Enter choice: 2

===== Admin Sign Up =====

Please enter the following details to create an admin account.

Enter username: Nolan

Enter password: 12345678

Confirm password: 12345678

=====

Press Enter to return to the admin menu...

===== Admin Access =====

1. Admin Login

2. Admin Sign Up

3. Exit

=====

Enter choice: 1

===== Admin Login =====

Please enter your admin credentials to access the library administration system.

Admin Username: Nolan

Admin Password: 12345678

Welcome, Administrator Nolan!

===== Admin Panel =====

1. Add a Book
2. Remove a Book
3. Borrow a Book
4. Return a Book
5. Back to Main Menu

=====

Enter your choice: 1

===== Add a Book =====

Please enter the book details below:

Enter Book Title: The Da Vinci Code

Enter Book Author: Dan Brown

Enter Book Genre: Historical Fiction

Quantity: 7

Enter Invoice Number: NF-25-1134

Enter Publisher Name: Doubleday

Enter Store Name: Whispering Pages

Enter Price: 961.44

Enter Purchase Date (YYYY-MM-DD): 2025-11-20

New book added successfully.

Book added successfully!

=====

Press Enter to return...

===== Admin Panel =====

1. Add a Book
2. Remove a Book
3. Borrow a Book
4. Return a Book
5. Back to Main Menu

=====

Enter your choice: 2

===== Remove Book =====

Enter the title of the book: The Da Vinci Code

Enter the author of the book: Dan Brown

Found 1 book(s):

1. Title: The da vinci code, Author: Dan brown, Genre: Historical fiction, Publisher: Whispering Pages

=====

Enter the number of the book to delete: 1

Are you sure you want to delete 'The da vinci code' (ID: 11)? (Y/N): Y

Book removed successfully.

=====

Press Enter to return...

```

===== Admin Panel =====
1. Add a Book
2. Remove a Book
3. Borrow a Book
4. Return a Book
5. Back to Main Menu
=====
Enter your choice: 3
Enter Member Username to borrow for: Nolan
===== Borrow a Book =====
1. Search by Title
2. Search by Author
3. Search by Genre
4. Search by Rating
5. Show all the books in Library
6. Back to Borrow/Return System
=====
Enter your choice (1-6): 1
Enter the title of the book: Jane Eyre
Title: Jane Eyre
Author: Charlotte Brontë«
Genre: Romance
Quantity: 1
Rating: 4.00
=====
Do You want to borrow a book from the above list?
Enter 'Y' or 'N': n
Press Enter to return...
===== Borrow a Book =====
1. Search by Title
2. Search by Author
3. Search by Genre
4. Search by Rating
5. Show all the books in Library
6. Back to Borrow/Return System
=====
Enter your choice (1-6): 2
Enter the author of the book: Harper Lee
Book is not Available.
Press Enter to return...
Press Enter to return...
===== Borrow a Book =====
1. Search by Title
2. Search by Author

```

3. Search by Genre
4. Search by Rating
5. Show all the books in Library
6. Back to Borrow/Return System

=====

Enter your choice (1-6): 3

Available Genres:

- 1.Dystopian
- 2.Romance
- 3.Fantasy
- 4.Historical Fiction
- 5.Coming-of-Age

Would you like to select from the above genres (Y/N)?y

Enter the No. of the genre of the book: 2

1 .

Title: Pride and Prejudice

Author: Jane Austen

Genre: Romance

Quantity: 3

Rating: 3.00

2 .

Title: Jane Eyre

Author: Charlotte Brontë

Genre: Romance

Quantity: 1

Rating: 4.00

=====

Do You want to borrow a book from the above list?

Enter 'Y' or 'N': y

Enter the serial numbers of the books you want to borrow separated by commas:

1

Selected Book:

Title: Pride and Prejudice

Author: Jane Austen

Pride and Prejudice is expected to be returned on 2026-02-01

A fee of ₹10 per day will be charged after 60 days.

Pride and Prejudice borrowed successfully!

=====

Press Enter to return...

Press Enter to return...

===== Borrow a Book =====

1. Search by Title
2. Search by Author
3. Search by Genre

4. Search by Rating
5. Show all the books in Library
6. Back to Borrow/Return System

=====

Enter your choice (1-6): 4

Rating of Books lies between 1 to 5

Enter the rating of the book: 5

1 .

Title: The Hobbit

Author: J.R.R. Tolkien

Genre: Fantasy

Quantity: 5

Rating: 5.00

2 .

Title: The Lord of the Rings

Author: J.R.R. Tolkien

Genre: Fantasy

Quantity: 3

Rating: 5.00

=====

Do You want to borrow a book from the above list?

Enter 'Y' or 'N': n

Press Enter to return...

===== Borrow a Book =====

1. Search by Title
2. Search by Author
3. Search by Genre
4. Search by Rating
5. Show all the books in Library
6. Back to Borrow/Return System

=====

Enter your choice (1-6): 5

Available books:

1 . 1984

2 . Pride and Prejudice

3 . The Hobbit

4 . War and Peace

5 . Jane Eyre

6 . The Catcher in the Rye

7 . The Lord of the Rings

=====

Do You want to borrow a book from the above list?

Enter 'Y' or 'N': n

===== Borrow a Book =====

1. Search by Title
2. Search by Author
3. Search by Genre
4. Search by Rating
5. Show all the books in Library
6. Back to Borrow/Return System

=====

Enter your choice (1-6): 6

===== Admin Panel =====

1. Add a Book
2. Remove a Book
3. Borrow a Book
4. Return a Book
5. Back to Main Menu

=====

Enter your choice: 4

Enter Member Username to return for: Nolan

===== Return a book =====

Returnable Books:

1. 1984 (Due: 2026-02-01)
2. Moby Dick (Due: 2026-02-01)
3. To Kill a Mockingbird (Due: 2026-02-01)
4. Jane Eyre (Due: 2026-02-01)
5. Pride and Prejudice (Due: 2026-02-01)

=====

Enter the number of the book to return: 5

Return the book 'Pride and Prejudice' and Pay ₹0.0!

Press Enter to Pay with Cash..

Would you like to rate the book? (Y/N):y

Rate the book (1 to 5): 5

Thank you for rating! The new average rating for this book is 4.00.

Book returned successfully!

=====

Press Enter to return..

===== Admin Panel =====

1. Add a Book
2. Remove a Book
3. Borrow a Book
4. Return a Book
5. Back to Main Menu

=====

Enter your choice: 5

===== Admin Access =====

1. Admin Login

2. Admin Sign Up

3. Exit

=====

Enter choice: 3

=== Library Management System ===

1. Login

2. Sign Up

3. Library Administration System

4. Exit

=====

Select an option (1-4): 4

Goodbye!

CHAPTER 8

TESTING

Testing is an essential part of software development that ensures the system functions correctly and meets user requirements. For this Library Management System, multiple testing approaches were applied to validate both functionality and code reliability.

Functional Testing

Functional testing focuses on verifying the features of the system according to the requirements. Tests performed include:

- ✓ User Signup and Login: Ensured that valid users can register and log in successfully, while invalid inputs are correctly rejected.
- ✓ Book Borrowing and Returning: Verified that users can borrow and return books, due dates are correctly calculated, and late fees are applied accurately.
- ✓ Admin Operations: Checked functionalities like adding and removing books, managing member transactions, and updating book information.

System Testing

System testing was performed to evaluate the integration of all modules. This included:

- ✓ Ensuring proper interaction between the user module, book management, and administration module.
- ✓ Verifying that database operations (like adding, updating, and retrieving records) execute correctly.
- ✓ Checking the system's behavior with edge cases, such as borrowing unavailable books or returning books late.

White Box Testing

White box testing was performed to examine the internal structure of the code. This included:

- ✓ Code Coverage: Ensuring that all major functions, conditional statements, and loops were executed at least once during testing.
- ✓ Transaction Verification: Confirming that all database transactions are correctly recorded, updated, and retrieved.

CHAPTER 9

HARDWARE AND SOFTWARE REQUIREMENTS

The Library Management System is a lightweight yet efficient application that requires a stable hardware and software environment to operate smoothly. It is designed to run exclusively on Windows. While the system is primarily software-based, adequate hardware resources are essential to ensure optimal performance, stability, and reliability.

For hardware, a quad-core processor (Intel Core i5 or higher) is recommended to handle database operations and multiple tasks efficiently, providing quick response times and smooth multitasking. The system requires a minimum of 4 GB RAM for basic execution of the program and MySQL database; however, 8 GB RAM or more is recommended for faster query execution and improved overall performance, particularly when other applications are running concurrently. In terms of storage, a minimum of 500 MB of free disk space is necessary for installation, with at least 10 GB recommended to accommodate database growth, backups, and additional project files. A Solid State Drive (SSD) is preferred over a traditional Hard Disk Drive (HDD) for faster data access and enhanced system responsiveness.

On the software side, the project is developed using Python 3.8 or above, leveraging its simplicity, modular structure, and extensive library support for database connectivity, file handling, and exception management. The backend database is MySQL 8.0 or later, which provides structured storage, efficient query execution, and reliable data integrity across multiple tables, including books, members, administration, and transactions. For development, Visual Studio Code is the preferred IDE due to its efficient interface, syntax highlighting, code completion, and debugging capabilities.

Together, these hardware and software specifications ensure that the Library Management System runs efficiently, reliably, and securely on Windows systems while providing a scalable platform for managing library operations.

CHAPTER 10

INSTALLATION PROCEDURE

To successfully install and run the Library Management System on a Windows environment, the following software components are required:

- **Python 3.8 or higher:** This programming language provides the runtime environment for executing the source code. Python is known for its simplicity, extensive library support, and compatibility with database-driven applications. It can be downloaded from www.python.org.
- **MySQL 9.5.0 (latest stable version):** MySQL serves as the backend database system for storing all library-related data, including books, members, and transaction records. Its structured storage, efficient query execution, and data integrity features ensure reliable performance. Download from www.mysql.org.

It is important to ensure that both Python and MySQL are properly installed and configured on the system before proceeding with the execution of the project.

Installation and Execution Steps

1. **Open the Project Folder:** Extract the Library Management folder to a preferred location on your Windows system. This folder contains all necessary source files required for running the application.
2. **Verify Software Installation:** Confirm that Python 3.8 or higher and MySQL 9.5.0 are installed correctly. Ensure that MySQL server is running and accessible.
3. **Locate the Main Script:** Within the project folder, identify the file `main.py`, which serves as the entry point of the application.
4. **Run the Main Script:** Execute `main.py` using a Python IDE (such as Visual Studio Code) or via the command line. The program will automatically initiate the setup process.

5. Automatic Database Setup: During the first execution, the application will create the required database, tables, and initial records in MySQL without any manual intervention. This ensures that all components are ready for immediate use.
6. Launching the Application: After the database setup is complete, the Library Management System will start, providing access to all its features including user registration, login, book management, borrowing, and returning functionalities.

BIBLIOGRAPHY

1. Sumita Arora , Computer Science with Python, Class 12, Dhanpat Rai & Co., 2023.
2. W3Schools, "Python Tutorial," <https://www.w3schools.com/python/>.
3. W3Schools, "MySQL Tutorial," <https://www.w3schools.com/mysql/>.
4. Programiz, "Python 3 Tutorial," <https://www.programiz.com/python-programming>.
5. GeeksforGeeks, "MySQL Database Tutorial," <https://www.geeksforgeeks.org/mysql-tutorial/>.
6. TutorialsPoint, Python 3 Programming Language, <https://www.tutorialspoint.com/python/index.htm>.
7. TutorialsPoint, MySQL Database Tutorial, <https://www.tutorialspoint.com/mysql/index.htm>.
8. Real Python, "Working with MySQL in Python," <https://realpython.com/python-mysql/>.
9. Stack Overflow Community, "Python and MySQL Programming Discussions". <https://stackoverflow.com/questions/tagged/python+mysql>
10. Michael Page. Software Development Life Cycle diagram.
MichaelPage.com.au.
https://www.michaelpage.com.au/sites/michaelpage.com.au/files/styles/advertice_node_desktop/public/2021-12/oct%20article%206%20SDLC_2.jpg.webp?itok=BfyS9uXJ